MATH 342W / 642 / RM 742 Spring 2026 HW #5

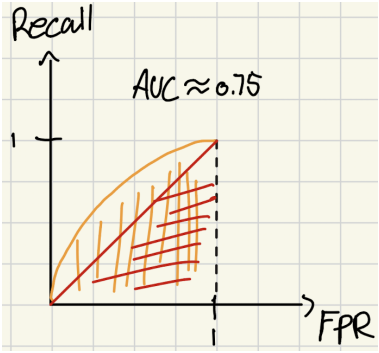
Isa Nawaz

Monday 11th May, 2026

Problem 1

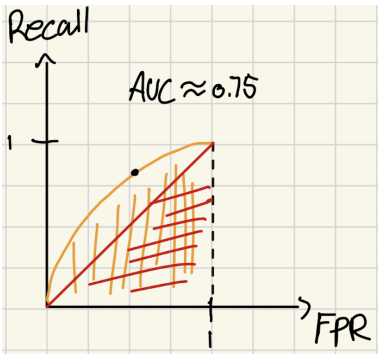
These are some questions related to probability estimation modeling and asymmetric cost modeling. It might be helpful to look back to the review the logistic regression before doing this problem.

(a) [easy] Graph a canonical ROC and label the axes. In your drawing estimate AUC. Explain very clearly what is measured by the x axis and the y axis.



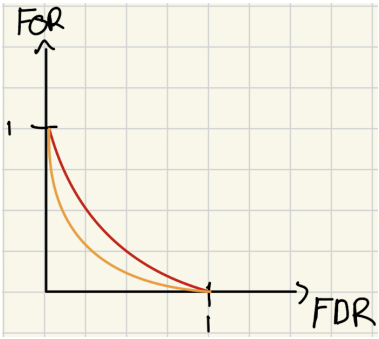
The x-axis is the False Positive Rate (FPR), which measures the proportion of actual negatives incorrectly predicted as positive.
The y-axis is Recall / True Positive Rate (TPR), which measures the proportion of actual positives correctly predicted as positive.
The ROC curve shows the tradeoff between true positives and false positives for different classification thresholds.

(b) [easy] Pick one point on your ROC curve from the previous question. Explain a situation why you would employ this model.



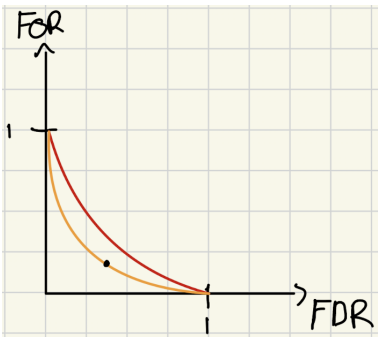
This point could be used for airport security because it is important to identify most dangerous items even if some harmless items are incorrectly flagged.

(c) [harder] Graph a FDR-FOR curve and label the axes. Explain very clearly what is measured by the x axis and the y axis.



The x-axis is the False Discovery Rate (FDR), which measures the proportion of positive predictions that are actually incorrect.
The y-axis is the False Omission Rate (FOR), which measures the proportion of negative predictions that are actually incorrect.
The curve shows the tradeoff between errors among positive predictions and errors among negative predictions.

(d) [easy] Pick one point on your FDR-FOR curve from the previous question. Explain a situation why you would employ this model.



The chosen point could be used for fire alarms. It is better for the alarm to go off by mistake sometimes than to miss a real fire.

(e) [difficult] [MA] The line of random guessing on the ROC curve is the diagonal line with slope one extending from the origin. What is the corresponding line of random guessing in the FDR-FOR curve? This is not easy...

The random guessing for the FDR-FOR curve would be the points representing the proportion of positives and negatives in the data.

Problem 2

These are some questions related to bias-variance decomposition. Assume the two assumptions from the notes about the random variable model that produces the δ values, the error due to ignorance.

(a) [easy] Write down (do not derive) the decomposition of MSE for a given $\boldsymbol{x}_*$ where $\mathbb{D}$ is assumed fixed but the response associated with $\boldsymbol{x}_*$ is assumed random.

$$\text{MSE}[x^*] = \sigma^2 + (Bias[x^*])^2$$

(b) [easy] Write down (do not derive) the decomposition of MSE for a given $\boldsymbol{x}_*$ where the responses in $\mathbb{D}$ is random but the $\boldsymbol{X}$ matrix is assumed fixed and the response associated with $\boldsymbol{x}_*$ is assumed random like previously.

$$\text{MSE}[x^*] = \sigma^2 + (Bias[G(x^*)])^2 + Var[G(x^*)]$$

(c) [easy] Write down (do not derive) the decomposition of MSE for general predictions of a phenomenon where all quantities are considered random.

$$\text{MSE} = \sigma^2 + E_{\vec{X}}\left[(Bias[G(x^*)])^2\right] + E_{\vec{X}}\left[Var[G(x^*)]\right]$$

(d) [difficult] Why is it in (a) there is only a "bias" but no "variance" term? Why did the additional source of randomness in (b) spawn the variance term, a new source of error?

In (a), $\mathbb{D}$ is fixed, so the model $g(x_*)$ is fixed. The only randomness comes from the future response Y_*. Since the model does not change, there is no variance term, only bias.
In (b), the responses in $\mathbb{D}$ are random, so different datasets produce different models. So, $G(x_*)$ becomes random, which creates the variance term.

(e) [harder] A high bias / low variance algorithm is underfit or overfit?

A high bias / low variance algorithm is underfit. The model is too simple, so it cannot capture the true relationship well.

(f) [harder] A low bias / high variance algorithm is underfit or overfit?

A low bias / high variance algorithm is overfit. The model is too complex and fits the training data too closely, causing predictions to vary a lot across datasets.

(g) [harder] Explain why bagging reduces MSE for "free" regardless of the algorithm employed.

Bagging reduces MSE because it averages many models together, reducing the variance term while keeping the bias the same.

(h) [harder] Explain why RF reduces MSE atop bagging M trees and specifically mention the target that it attacks in the MSE decomposition formula and why it's able to reduce that target.

Random forests reduce MSE further by reducing the variance term even more. RF does this by making the trees less correlated through random feature selection at each split. Lower correlation between trees improves the averaging effect, which reduces variance further.

(i) [difficult] When can RF lose to bagging M trees? Hint: think hyperparameter choice.

RF can lose to bagging if the number of features considered at each split is too small. Then the trees become too weak and the model gains too much bias. The decrease in variance may not be enough to make up for this.

Problem 3

These are some questions related to missingness.

(a) [easy] [MA] What are the three missing data mechanisms? Provide an example when each occurs (i.e., a real world situation). We didn't really cover this in class so I'm making it a MA question only. This concept will NOT be on the exam.

1. MCAR - Missing Completely at Random. The missingness is not related to anything in the dataset. A student accidentally skips a question on a test because it was at the bottom.
2. MAR - Missing at Random The missingness depends on another observed variable. College students are less likely to answer phone call surveys.
3. NMAR - Not Missing at Random The missingness depends on the value itself. People with low GPAs avoid reporting their GPA.

(b) [easy] Why is listwise-deletion a *terrible* idea to employ in your $\mathbb{D}$ when doing supervised learning?

Because any row with any missingness gets dropped, estimation error increases. Selection bias gets introduced, deleting a part of X space. Generalization error increases because you extrapolate more in the future for MAR/NMAR.

(c) [easy] Why is it good practice to augment $\mathbb{D}$ to include missingness dummies? In other words, why would this increase oos predictive accuracy?

To track where the missing data was when imputing. Also the fact that a value is missing can also be used as data.

(d) [easy] To impute missing values in $\mathbb{D}$, what is a good default strategy and why?

A good default strategy is to fill in the missing values with the mean/mode of the respective column each missing value is in.

Problem 4

These are some questions related to gradient boosting. The final gradient boosted model

after M iterations is denoted G_M which can be written in a number of equivalent ways (see below). The g_t's denote constituent models and the G_t's denote partial sums of the constituent models up to iteration number t. The constituent models are "steps in functional steps" which have a step size η and a direction component denoted $\tilde{g}_t$. The directional component is the base learner $\mathcal{A}$ fit to the negative gradient of the objective function L which measures how close the current predictions are to the real values of the responses:

$$\begin{aligned}
G_M &= G_{M-1} + g_M \\
&= g_0 + g_1 + \ldots + g_M \\
&= g_0 + \eta\tilde{g}_1 + \ldots + \eta\tilde{g}_M \\
&= g_0 + \eta\mathcal{A}\left(\langle \boldsymbol{X}, -\nabla L(\boldsymbol{y}, \hat{\boldsymbol{y}}_1)\rangle, \mathcal{H}\right) + \ldots + \eta\mathcal{A}\left(\langle \boldsymbol{X}, -\nabla L(\boldsymbol{y}, \hat{\boldsymbol{y}}_M)\rangle, \mathcal{H}\right) \\
&= g_0 + \eta\mathcal{A}\left(\langle \boldsymbol{X}, -\nabla L(\boldsymbol{y}, g_1(\boldsymbol{X}))\rangle, \mathcal{H}\right) + \ldots + \eta\mathcal{A}\left(\langle \boldsymbol{X}, -\nabla L(\boldsymbol{y}, g_M(\boldsymbol{X}))\rangle, \mathcal{H}\right)
\end{aligned}$$

(a) [easy] From a perspective of only multivariable calculus, explain gradient descent and why it's a good idea to find the minimum inputs for an objective function L (in English).

The gradient points in the direction where a function increases the most. The negative gradient points in the direction where it decreases the most. Gradient descent repeatedly takes small steps in the negative gradient direction to make the objective function L smaller. This helps reduce prediction error.

(b) [easy] Write the mathematical steps of gradient boosting for supervised learning below. Use L for the objective function to keep the procedure general. Use notation found in the problem header.

Start at $G_0 = g_0$
Then
$\Delta g_1 = \mathcal{A}\left(\boldsymbol{X}, -\nabla L(\vec{y}, \hat{\vec{y}}_0), \mathcal{H}\right)$
$G_1 = g_0 + \eta\Delta g_1 = g_0 + \eta\mathcal{A}\left(\boldsymbol{X}, -\nabla L(\vec{y}, \hat{\vec{y}}_0), \mathcal{H}\right)$
$\Delta g_2 = \mathcal{A}\left(\boldsymbol{X}, -\nabla L(\vec{y}, \hat{\vec{y}}_1), \mathcal{H}\right)$
$G_2 = G_1 + \eta\Delta g_2 = G_1 + \eta\mathcal{A}\left(\boldsymbol{X}, -\nabla L(\vec{y}, \hat{\vec{y}}_1), \mathcal{H}\right)$
$\vdots$
$\Delta g_t = \mathcal{A}\left(\boldsymbol{X}, -\nabla L(\vec{y}, \hat{\vec{y}}_{t-1}), \mathcal{H}\right)$
$G_t = G_{t-1} + \eta\Delta g_t = G_{t-1} + \eta\mathcal{A}\left(\boldsymbol{X}, -\nabla L(\vec{y}, \hat{\vec{y}}_{t-1}), \mathcal{H}\right)$
Final model:
$G_M = g_0 + g_1 + \cdots + g_M$

(c) [easy] For regression, what is $g_0(\boldsymbol{x})$?

The initial prediction is the average of all response values.

(d) [easy] For probability estimation for binary response, what is $g_0(\boldsymbol{x})$?

The initial prediction is the proportion of 1's in the training data.

(e) [harder] What are all the hyperparameters of gradient boosting? There are more than just two.

The number of iterations M, the algorithm $\mathcal{A}$, the hypothesis set $\mathcal{H}$, and the complexity of the model.

(f) [easy] For regression, rederive the negative gradient of the objective function L.

$$L(\vec{y}, \hat{\vec{y}}) = SSE = (y_1 - \hat{y}_1)^2 + \cdots + (y_n - \hat{y}_n)^2$$

$$\nabla L(\vec{y}, \hat{\vec{y}}) = \begin{bmatrix} -2(y_1 - \hat{y}_1) \\ -2(y_2 - \hat{y}_2) \\ \vdots \\ -2(y_n - \hat{y}_n) \end{bmatrix} = -2\vec{e}$$

$$-\nabla L(\vec{y}, \hat{\vec{y}}) = 2\vec{e}$$

(g) [easy] For probability estimation for binary response, rederive the negative gradient of the objective function L.

$L(\vec{y}, \hat{\vec{y}}) = \prod_{i=1}^{n} \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1-y_i}$
Using the logistic link:
$\hat{p}_i = \frac{e^{\hat{y}_i}}{1+e^{\hat{y}_i}}$
Substitute into the likelihood:
$L(\vec{y}, \hat{\vec{y}}) = \prod_{i=1}^{n} \frac{e^{\hat{y}_i y_i}}{1+e^{\hat{y}_i}}$
Take the log:
$\ln(L) = \sum_{i=1}^{n} \left(\hat{y}_i y_i - \ln(1 + e^{\hat{y}_i}) \right)$

$$\nabla L(\vec{y}, \hat{\vec{y}}) = \begin{bmatrix} y_1 - \hat{p}_1 \\ y_2 - \hat{p}_2 \\ \vdots \\ y_n - \hat{p}_n \end{bmatrix} = \vec{y} - \hat{\vec{p}}$$

$-\nabla L(\vec{y}, \hat{\vec{y}}) = \hat{\vec{p}} - \vec{y}$

(h) [difficult] For probability estimation for binary response scenarios, what is the unit of the output $G_M(\boldsymbol{x}_\star)$?

For probability estimation with binary response, the output $G_M(\boldsymbol{x}_\star)$ represents a probability estimate, so its units are probabilities between 0 and 1.

(i) [easy] For the base learner algorithm $\mathcal{A}$, why is it a good idea to use shallow CART (which is the recommended default)?

It is a good idea to use shallow CART because shallow trees are simple and less likely to overfit. Each tree only learns a small part of the pattern, and gradient boosting combines many of them together to improve predictions.

(j) [difficult] For the base learner algorithm $\mathcal{A}$, why is it a bad idea to use deep CART?

Deep trees can overfit the training data. They have very low bias but very high variance, so the boosted model may memorize noise and perform worse out-of-sample.

(k) [difficult] For the base learner algorithm $\mathcal{A}$, why is it a bad idea to use OLS for regression (or logistic regression for probability estimation for binary response)?

Using OLS or logistic regression is a bad idea because they are already powerful models. Gradient boosting works better with weak learners that only learn small parts of the pattern at a time. Strong models can overfit the data too quickly.

(l) [difficult] If M is very, very large, what is the risk in using gradient boosting even using shallow CART as the base learner (the recommended default)?

> If M becomes very large, the model keeps adding more and more trees and may begin fitting random noise instead of the true pattern. This causes overfitting, increases variance, and can hurt out-of-sample prediction accuracy.

(m) [difficult] If η is very, very large but M reasonably correctly chosen, what is the risk in using gradient boosting even using shallow CART as the base learner (the recommended default)?

> If η is very large, each step in gradient boosting becomes too large. The model may over-correct errors instead of improving gradually.